

ECOM EXPERTS

Ecom iatracking

Manual de usuario, tecnico y de desarrollo

ecom_iatracking 1.0.0 · 07/09/2026

Contenido

1. Instalacion

2. Configuracion

3. Uso

4. Runbook

5. Privacidad (RGPD)

1. Arquitectura general

2. Hooks

3. Base de datos

4. Endpoints externos

17TRACK (Ecomlatracking17TrackClient)

Proveedores IA

5. Flujo de cache (lazy, cache-or-API)

6. Seguridad y privacidad

1. Convenciones obligatorias

2. Clases principales

3. Capa IA — anadir un proveedor

4. Self-heal de schema

5. Traducciones

6. Registro Gmartos

7. Reglas anti-desastre (BLOQUEANTES)

Seguimiento de envios con 17TRACK, consulta publica, timeline en la cuenta del cliente, panel en la ficha de pedido y asistente IA de transporte opcional.

1. Instalacion

1. Sube el ZIP del modulo desde **Modulos > Gestor de modulos > Subir un modulo** e instala. Se crean las tablas, los hooks, la pestana de administracion y las URLs amigables `track-order` y `my-orders-tracking`.
2. Abre la configuracion del modulo. Todo el mantenimiento (columnas, hooks, purga) se auto-repara al entrar aqui.

Importante: las llamadas salientes a 17TRACK vienen **desactivadas de fabrica** (kill-switch de seguridad). El modulo funciona y muestra datos cacheados, pero no contacta 17TRACK hasta que un desarrollador levante el kill-switch en el codigo. Esto evita enviar peticiones por pedidos antiguos al instalar.

2. Configuracion

Pestanas del formulario:

- **17TRACK API:** cuota restante (solo visible con el kill-switch levantado y la key puesta), **17TRACK API key** (tu 17token), version de API (def. `v2.4`) y TTL de cache en horas (def. 6).
- **Behaviour:** estados de pedido que activan el seguimiento (multi-seleccion + palabras clave shipped/delivered/paid), dias de gracia tras la entrega y dias de retencion antes de purgar. El cron de mantenimiento no se incluye en esta version (el interruptor solo prepara el token).
- **Security & privacy:** limite de consultas de la puerta publica por IP/hora, confianza en cabeceras de proxy, captcha en la puerta publica, traduccion de descripciones y log de depuracion (escribe `.log` en `/logs`).
- **Display:** mostrar el boton flotante de seguimiento y su icono (paquete por defecto o avion).
- **AI assistant:** activar el asistente, elegir proveedor (sin proveedor seleccionado la IA queda apagada aunque haya key), API key, Base URL (Ollama/Otro), modelo (vacio = fallback seguro por proveedor), limites de tokens/cache/chat, documentos de conocimiento (arrastrar y soltar PDF/TXT/DOC/DOCX, max 15 MB), paginas CMS a ingerir, "Solo extraccion local (RGPD)" y log de depuracion IA.
- **Carrier mapping:** asocia cada transportista de PrestaShop a su codigo numerico 17TRACK. La columna "Sugerido" se rellena por auto-deteccion; el boton "Forzar re-deteccion" limpia las sugerencias.

Guarda con **Save**. La configuracion se re-lee tras guardar para confirmar la persistencia (multi-tienda soportado).

3. Uso

- **Boton flotante:** si esta activo, aparece en toda la tienda y abre la puerta publica de seguimiento.

- **Puerta publica** (`track-order`): el cliente introduce referencia del pedido y email del checkout; se valida la propiedad y se muestra el estado, el stepper de hitos, la ETA, la ultima ubicacion y el chat IA (si esta activo).
- **Mis pedidos** (`my-orders-tracking`): desde la cuenta del cliente, lista sus pedidos con seguimiento y su timeline.
- **Ficha de pedido (back-office)**: panel de tracking con numero, estado, posicion y boton de refrescar seguimiento.

4. Runbook

- **Cuota agotada / API caida**: la tienda no da error; sirve el ultimo estado guardado con aviso. Revisa la cuota en la pestana 17TRACK API.
- **Kill-switch**: mientras este activo no hay llamadas a 17TRACK. Lo levanta un desarrollador en el codigo (`EcomIatracking17TrackClient::KILL_SWITCH = false`) cuando confirmes que quieres empezar a rastrear en produccion.
- **Seed inicial**: al instalar, los pedidos preexistentes se marcan como ya procesados; el modulo nunca dispara acciones por historicos.
- **Asistente IA no responde**: comprueba que esta activado, que hay proveedor seleccionado y key valida; si la cuota del proveedor se agota, el asistente se degrada y se puede reactivar volviendo a guardar una key fresca.

5. Privacidad (RGPD)

- Datos de tracking gestionados con 17TRACK como sub-procesador; el asistente IA usa el proveedor que elijas (OpenAI, Gemini, Anthropic, Ollama u otro compatible) como sub-procesador. Informa de ambos en tu politica de privacidad.
- Con **Solo extraccion local** activo, los ficheros crudos nunca se suben al proveedor IA: solo se usa el texto extraido localmente.
- El registro de chat guarda la PII hasheada y se purga por el periodo de retencion. El borrado RGPD del cliente elimina sus datos asociados.
- En la puerta publica solo se muestra el estado del envio; los datos del cliente se mantienen privados.
- El modulo envia semanalmente el email de la tienda (PS_SHOP_EMAIL) y su dominio a modules.gmartos.es para registrar la instalacion y comprobar actualizaciones.

Ecom iatracking — Documentacion tecnica

Modulo PrestaShop 1.7 / 8 / 9 de seguimiento de envios con 17TRACK, puerta de consulta publica, timeline en la cuenta del cliente y asistente IA de transporte opcional. Clase principal `Ecom_Iatracking` (namespace global, sin `declare(strict_types)`), compatible PHP 7.4 y 8.x, sin Composer (cURL nativo).

1. Arquitectura general

```
ecom_iatracking.php      Clase principal: hooks, config (HelperForm),
                        fetchTracking() (cache-or-API), factory IA,
                        registro Gmartos.

classes/
  EcomIatrackingInstaller.php  install/uninstall, schema, self-heal columnas,
                                addnewfeatures(), fixMainTabIntegrity(), purga.
  EcomIatrackingLogger.php     Log condicional a /logs (flags DEBUG).
  EcomIatracking17TrackClient.php  Cliente HTTP 17TRACK. const KILL_SWITCH=true.
  EcomIatrackingStatusMapper.php  Mapea la respuesta 17TRACK a un DTO interno.
  EcomIatrackingStepper.php     Construye el modelo del stepper de estados.
  EcomIatrackingStatusMapper / Stepper  Normalizan estado + milestones + ETA.
  EcomIatrackingRateGate.php    Rate-limit por IP (puerta publica y chat).
  EcomIatrackingOwnershipValidator.php  Valida referencia+email / propiedad.
  EcomIatrackingShipment.php    ObjectModel del envio.
  EcomIatrackingEvent.php       ObjectModel del evento de tracking.
ai/
  EcomIatProviderInterface.php  Contrato de proveedor IA.
  providers/EcomIatAbstractProvider.php  Base cURL comun.
  providers/{OpenAi,Gemini,Anthropic,Ollama,GenericOpenAi}Provider.php
  EcomIatAssistant.php         Orquesta pregunta -> contexto -> proveedor.
  EcomIatKnowledgeManager.php  Ingesta KB (docs subidos + paginas CMS).
  EcomIatFileExtractor.php     Extraccion local de texto (PDF/TXT/DOC/DOCX).
  EcomIatPromptBuilder.php     Construye el prompt (PII enmascarada).
  EcomIatCache.php             Cache de contexto/respuesta IA.
  EcomIatUsage.php             Cuotas, breaker de cuota/auth, contadores.
controllers/
  front/tracking.php           EcomIatrackingTrackingModuleFrontController (puerta publica).
  front/myorders.php           EcomIatrackingMyordersModuleFrontController (mis pedidos).
  front/ajax.php               EcomIatrackingAjaxModuleFrontController (refresh + chat).
  admin/AdminEcomIatrackingController.php  Tab AdminEcomIatracking.
  admin/AdminEcomIatKbController.php       Endpoint AJAX del dropzone KB.
views/  plantillas .tpl, css, js (assets locales, sin CDN, sin ?v=).
translations/en-US/  ficheros .xlf (sistema nuevo de traduccion).
```

2. Hooks

Hook	Uso
<code>actionValidateOrder</code>	Crea la fila de envio si el pedido nace en un estado seguido con numero de tracking.
<code>actionOrderStatusPostUpdate</code>	Crea/actualiza la fila de envio al entrar el pedido en un estado seguido.
<code>displayHeader</code>	Inyecta CSS/JS del front y del boton flotante.
<code>displayFooterAfter</code> / <code>displayFooter</code>	Renderiza el boton flotante (guard <code>\$fabRendered</code> para emitirlo una sola vez).
<code>displayCustomerAccount</code>	Enlace "Mis pedidos - seguimiento" en la cuenta.
<code>displayAdminOrderMain</code>	Panel de tracking en la ficha de pedido del back-office.
<code>actionGetAdminOrderButtons</code>	Boton de refresco de tracking en la ficha de pedido.
<code>moduleRoutes</code>	URLs amigables <code>track-order</code> y <code>my-orders-tracking</code> .
<code>actionDeleteGDPRCustomer</code>	Borrado RGPD de datos asociados al cliente.

3. Base de datos

Todas con prefijo `_DB_PREFIX_` . 'ecom_iatracking_' :

Tabla	Contenido
<code>ecom_iatracking_shipment</code>	Una fila por (pedido, numero, transportista): estado, sub-estado, ETA, hashes, <code>raw_json</code> , flags <code>is_registered</code> / <code>is_terminal</code> , <code>last_synced_at</code> .
<code>ecom_iatracking_event</code>	Timeline detallado por envio (hora, etapa, descripcion, pais/ciudad, lat/long, proveedor).
<code>ecom_iatracking_carrier_map</code>	Mapeo transportista PrestaShop -> codigo numerico 17TRACK (<code>is_suggested</code> marca auto-deteccion). Scope por <code>id_shop</code> .
<code>ecom_iatracking_cache</code>	Cache de respuestas de tracking.
<code>ecom_iatracking_attempts</code>	Intentos de la puerta publica (rate-limit por IP).
<code>ecom_iatracking_apilog</code>	Log de llamadas a la API 17TRACK.
<code>ecom_iatracking_salt</code> / <code>ecom_iatracking_kb</code>	Salt de hashing y base de conocimiento IA (fragmentos + metadatos).
<code>ecom_iatracking_chatlog</code>	Preguntas/respuestas del chat (solo con <code>ECOMIAT_DEBUG</code> , con PII hasheada).
<code>ecom_iatracking_ctxcache</code>	Cache de contexto IA por pregunta.
<code>ecom_iatracking_usage</code>	Contadores de uso IA + estado del breaker.

Versionado de schema en `ECOM_IATRACKING_DB_VERSION` / `ECOMIAT_KB_VERSION`. El self-heal (`INFORMATION_SCHEMA` + `ALTER TABLE`) corre en `install()`, `addnewfeatures()` y el constructor del `ObjectModel`, de modo que una columna nueva existe antes de que cualquier `SELECT` del modelo la use.

4. Endpoints externos

17TRACK (`EcomIatracking17TrackClient`)

Base `https://api.17track.net/track/{version}` (por defecto `v2.4`). Metodos: `register`, `gettrackinfo`, `getquota`, `changecarrier`, `deletetrack`. Autenticacion por cabecera `17token`. `const KILL_SWITCH = true`: mientras este activo, cada punto de envio hace `return null` sin tocar la red. La cuota se lee por `/getquota` y se muestra en la configuracion.

Proveedores IA

Endpoint segun proveedor: OpenAI, Google Gemini

(`https://generativelanguage.googleapis.com/v1beta/models`), Anthropic, Ollama (self-host) y "Otro" (compatible OpenAI). Sin proveedor seleccionado (`ECOMIAT_PROVIDER=''`) la IA queda desactivada aunque haya key. Modelo por defecto seguro cuando el campo esta vacio (Gemini: `gemini-flash-latest`).

5. Flujo de cache (lazy, cache-or-API)

`fetchTracking($idOrder, $userAction)` : 1. Carga las filas de envio del pedido; si no hay, devuelve estado `pending` . 2. Por cada fila: si esta fresca dentro del TTL (`ECOM_IATRACKING_CACHE_TTL_HOURS` , def. 6h) o es terminal, sirve la cache. 3. Si hay accion de usuario, esta obsoleta y hay numero real: `refreshFromApi()` -> `register-once` (guard estatico + claim atomico por UPDATE condicional) -> `gettrackinfo` -> mapea -> persiste columnas y eventos. Con KILL_SWITCH las llamadas devuelven null y se sigue sirviendo la cache (marca `stale`). 4. Un 429/cuota agotada NO produce 500: sirve la cache antigua con aviso. 5. El estado global del pedido es el menos avanzado de sus paquetes (multi-parcel seguro).

6. Seguridad y privacidad

- KILL_SWITCH de fabrica en el cliente 17TRACK; seed en `install()` (`ECOM_IATRACKING_SEEDED`) para no reprocesar historicos; sin loops sobre tablas historicas; sin cron activo en v1 (controlador no incluido; solo se prepara el token `ECOM_IATRACKING_CRON_TOKEN`).
- Puerta publica: validacion referencia+email (`OwnershipValidator`), rate-limit por IP (`ECOM_IATRACKING_RATE_LIMIT`), captcha opcional, lectura de IP tras proxy solo si `ECOM_IATRACKING_TRUST_PROXY` .
- IA: PII enmascarada antes del prompt; `ECOMIAT_LOCAL_ONLY` impide subir el fichero crudo al proveedor (solo texto extraido localmente); `finfo` en las subidas; chatlog con PII hasheada y purga por retencion; breaker de cuota/auth.
- Traducciones por sistema nuevo (`isUsingNewTranslationSystem()=true`), dominios `Modules.Ecomiatracking.Shop / .Admin` , sin `mod=` . Smarty con escape en toda variable, sin `{displayPrice}` , precios formateados en PHP.
- Registro de la instalacion en `modules.gmartos.es` (ping semanal silencioso, cURL nativo, timeout 3s, SSL verificado).

Ecom iatracking — Guia de desarrollo

Convenciones y puntos de extension para trabajar sobre el modulo sin romper la compatibilidad PS 8/9 ni las reglas del proyecto.

1. Convenciones obligatorias

- Clase principal `Ecom_Iatracking` en `ecom_iatracking.php`, namespace global, sin `namespace` ni `declare(strict_types)`. Carga de clases con `require_once`.
- Cabecera en TODOS los ficheros (`.php/.js/.css/.tpl/.sql`): autor "Ecom Experts", email `ecomseo@gmail.com`, copyright 2026, licencia AFL-3.0. En PHP: `<?php` seguido inmediatamente del bloque `/**` (cero lineas en blanco) y luego `if (!defined('_PS_VERSION_')) { exit; }`. Los `.xml` son XML puro (sin cabecera PHP).
- `index.php` en cada directorio (cabecera + `header('Location: ../'); exit;`).
- PHP 7.4 y 8.x: nunca `$var::class` (usar `is_object($x) ? get_class($x) : ''`); `\Context::getContext()` con backslash; sin Composer/Guzzle (cURL nativo).
- Nombres EXACTOS de front controllers (un desajuste = 404):
`EcomIatrackingTrackingModuleFrontController`, `EcomIatrackingMyordersModuleFrontController`, `EcomIatrackingAjaxModuleFrontController`. Admin: `AdminEcomIatrackingController` (`Tab AdminEcomIatracking`) y `AdminEcomIatKbController`. Nunca `Tab::getIdFromClassName` (PS9 `NonUniqueResult`): usar `Db::getValue` + `fixMainTabIntegrity()`.
- Prefijos de config: `ECOM_IATRACKING_*` (tracking) y `ECOMIAT_*` (IA).

2. Clases principales

- `EcomIatrackingInstaller`: `install()`, `uninstall()`, creacion de tablas (`sql/`), registro de hooks, `addnewfeatures()` (mantenimiento idempotente: self-heal de columnas via `INFORMATION_SCHEMA`, re-aseccion de hooks, purga por retencion) y `fixMainTabIntegrity()` (dedupe de tabs por SQL).
- `EcomIatracking17TrackClient`: cliente HTTP. `const KILL_SWITCH = true`. Cada metodo publico (`register`, `getTrackInfo`, `getQuota`, `changeCarrier`, `deleteTrack`) corta con `if (self::KILL_SWITCH) { return null; }` al inicio.
- `EcomIatrackingStatusMapper`: `map($trackInfo, $rawEnvelope) -> DTO` (estado, sub-estado, ETA, origen/destino, timeline, hashes, terminal).
- `EcomIatrackingStepper`: convierte el DTO en el modelo del stepper visual.
- `EcomIatrackingRateGate` y `EcomIatrackingOwnershipValidator`: rate-limit por IP y validacion de propiedad del pedido.
- `EcomIatrackingShipment` / `EcomIatrackingEvent`: `ObjectModel` con `$definition` tipado; las columnas nuevas deben migrarse (self-heal) antes de que el modelo cargue.

3. Capa IA — anadir un proveedor

1. Crear `classes/ai/providers/MiProvider.php` extendiendo `EcomIatAbstractProvider` e implementando `EcomIatProviderInterface` (cabecera + `if (!defined('_PS_VERSION_')) { exit; } + require_once`).
2. Implementar la llamada con cURL nativo reutilizando los helpers de la clase base (timeout, parseo de error de cuota/auth para el breaker).
3. Registrar la opcion en `getConfigForm()` (select `ECOMIAT_PROVIDER`) y en el factory `getAiProvider()` de la clase principal (nuevo `case`).
4. Anadir su `require_once` en `requireAiClasses()`.
5. Si necesita key: dejar que el factory devuelva null cuando falte (los hosted la exigen; Ollama/other pueden funcionar sin ella). Modelo por defecto seguro cuando `ECOMIAT_MODEL` este vacio.

El asistente (`EcomIatAssistant`) orquesta: gates de uso (breaker + presupuestos) -> contexto de KB (`EcomIatKnowledgeManager` + `EcomIatCache`) -> prompt con PII enmascarada (`EcomIatPromptBuilder`) -> proveedor -> normalizacion; en cuota/auth dispara `tripBreaker()`, en errores de red/formato falla en blando.

4. Self-heal de schema

Toda columna nueva se migra en una funcion guardada por flag de `Configuration`, invocada en `install()`, `addnewfeatures()` y el self-heal del constructor del `ObjectModel`. Patron: comprobar `INFORMATION_SCHEMA.COLUMNS` y hacer `ALTER TABLE` solo si falta. Nunca `LIMIT 1` manual en `getRow` / `getValue`; `pSQL()` en SQL manual; limpieza con `IF EXISTS` / try-catch.

5. Traducciones

- Extraccion determinista: `grep` / regex sobre `{l s='...'} en .tpl` y `->trans('...', [...], 'Dominio')` en `.php`, diff contra los `resname` del `.xlf`. NO usar agentes LLM para esto.
- `.tpl`: `{l s='...' d='Modules.Ecomiatracking.Shop'}` (front) o `.Admin` (back). **Prohibido** `mod='ecom_iatracking'` (legacy, ignora el `.xlf`).
- PHP: `$this->trans('...', [], 'Modules.Ecomiatracking.Admin')`; en controllers `$this->module->getTranslator()->trans(...)`. Solo strings literales.
- Ficheros: `translations/en-US/ModulesEcomiatrackingShop.en-US.xlf` y `ModulesEcomiatrackingAdmin.en-US.xlf`, XLIFF 1.2, `source == target` (ingles), match por `<source>` exacto (respeta `...`, `-`, `&`, comillas). Regenerar el `.xlf` tras anadir cualquier string nuevo.

6. Registro Gmartos

`checkGmartosUpdate()` (privada, cURL nativo, timeout 3s, un ping semanal, SSL verificado) registra la instalacion en `modules.gmartos.es` y avisa si hay version nueva. Se invoca en `install()` (forzando `GMARTOS_LAST_PING_*=0`) y al inicio de `getContent()` (alerta via `views/templates/admin/gmartos_update.tpl`). `uninstall()` limpia las claves `GMARTOS_*`.

7. Reglas anti-desastre (BLOQUEANTES)

- Ningun scan de tablas historicas para acciones externas. Ventana maxima de reprocesado 24h. Seed obligatorio antes de cualquier scan.
- KILL_SWITCH del 17TRACK de fabrica en `true` ; el merchant no lo cambia (solo en codigo). Cron no incluido en v1.
- Sin CDN, assets locales, sin `?v=` en URLs de assets, escape de toda variable Smarty, `finfo` en subidas, PII enmascarada antes del prompt IA.